Le contexte du projet aide la génération

```
package org.apache.commons.math3.stat.regression;
import org.apache.commons.math3.linear.LUDecomposition;
public class GLSMultipleLinearRegression extends AbstractMultipleLinearRegression {
    private RealMatrix Omega;
    private RealMatrix OmegaInverse;

    protected RealVector calculateBeta();
    protected RealMatrix calculateBetaVariance();
}
```

Imports

Champs

Signatures

Fig. 2(a), extrait adapté : annotations traduites ; ce n'est pas le prompt complet.

CodeLlama-13B

Génération de fonctions (FCG)

Trois prompts, un même modèle.

Prompt	**pass-all@1**
Un exemple	12,84 %
Quatre exemples	14,50 %
Contexte du projet	21,91 %

Source : Zeng et al., ISSTA 2024, fig. 2(a) et tableau 3.

Un écart plus marqué en réparation

FCG : générer une fonction

APR : réparer une fonction

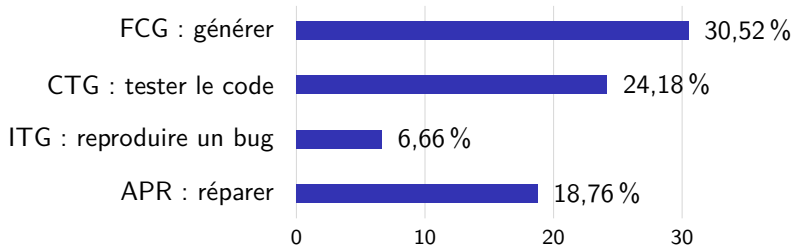
Modèle	**FCG**	**APR**
CodeLlama-34B	22,82 %	5,01 %
GPT-3.5-Turbo	23,37 %	9,31 %
GPT-4	**30,52 %**	**18,76 %**

Écart GPT-4 / CodeLlama : **7,70 points** en FCG, **13,75** en APR.

pass-all@1. Source : Zeng et al., tableau 4.
Versions : GPT-3.5-Turbo-0301 et GPT-4-0314.

GPT-4 : des scores différents selon la tâche

GPT-4-0314



Réussite : FCG / APR : passer les tests.
CTG : test exécutable couvrant le code.
ITG : test en échec avec le bug, puis réussi après correction.

Source : Zeng et al., §3.5 et tableau 4.

Détecter les bugs reste difficile

940	50 %	52,16 %
fonctions	réponse constante	GPT-4

470 fonctions avec un bug et 470 versions corrigées.

Accuracy : proportion de réponses correctes.
GPT-4 dépasse la référence de **2,16 points**.

À retenir

Le contexte aide la génération. Les autres tâches restent à vérifier séparément.

Source : Zeng et al., §3.3 et tableau 4 ; GPT-4-0314.